

[DB] POSTGRESQL PRACTICE WORKBOOK

DVD Rental Database

Basic —> Advanced —> Expert

“From `SELECT *` to query wizard — one rental at a time.”

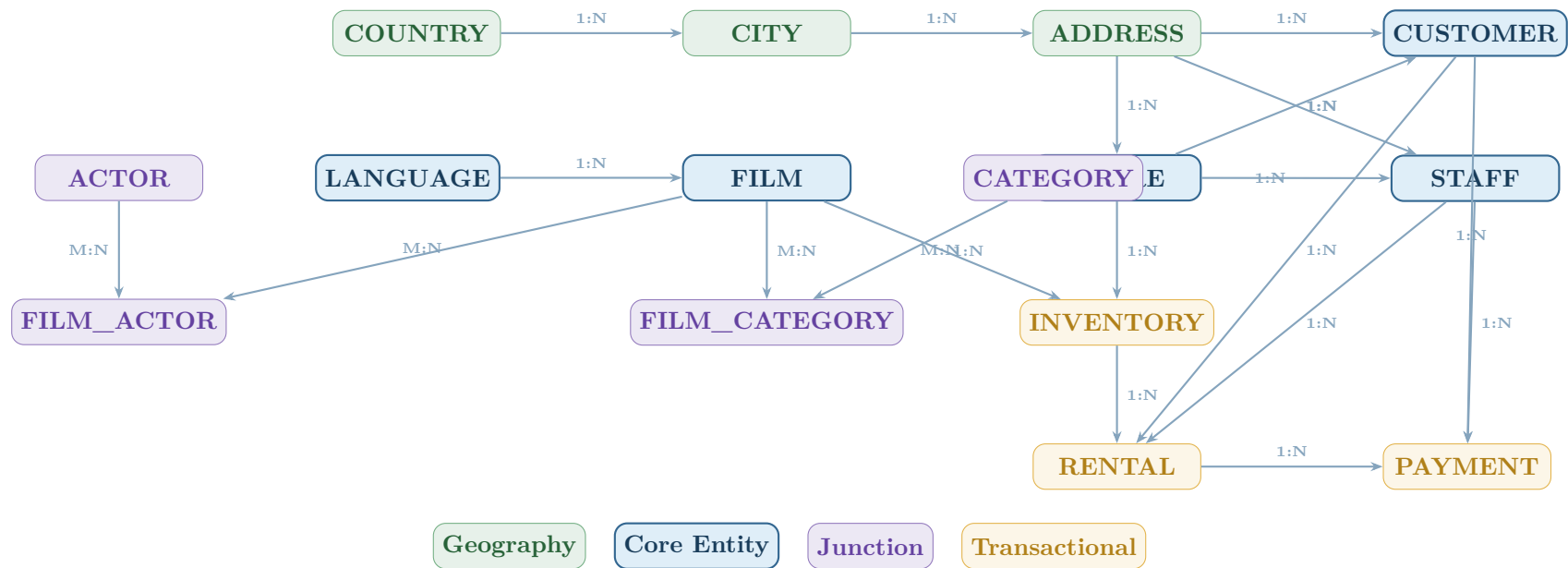
Muhammad Usman

Data & Cloud Architecture

21 Topics • 105 Questions • SQL
Hints & Answers • Concept Guides

■ Easy ■ Medium ■ Hard ■ Expert

[DB] DVD Rental — Entity Relationship Diagram



[DB] Schema Quick Reference

Table	Key Columns
film	film_id PK, title, description, release_year, language_id FK, rental_duration, rental_rate , length, replacement_cost, rating , fulltext (tsvector)
actor	actor_id PK, first_name, last_name
category	category_id PK, name
language	language_id PK, name
inventory	inventory_id PK, film_id FK, store_id FK
rental	rental_id PK, rental_date, inventory_id FK, customer_id FK, return_date , staff_id FK
payment	payment_id PK, customer_id FK, staff_id FK, rental_id FK, amount , payment_date
customer	customer_id PK, store_id FK, first_name, last_name, email , address_id FK, active
staff	staff_id PK, first_name, last_name, store_id FK, email, active
store	store_id PK, manager_staff_id FK, address_id FK
address	address_id PK, address, city_id FK, postal_code
city	city_id PK, city, country_id FK
country	country_id PK, country

Contents

1	Introduction to SQL	3
2	Data Definition Language (DDL)	4
3	Data Manipulation Language (DML)	5
4	Basic SQL Queries	6
5	Sorting and Limiting Results	7
6	Functions and Expressions	7
7	Grouping and Aggregating Data	8
8	Joins	9
9	Advanced Joins and Set Operations	10
10	Subqueries and Nested Queries	10
11	Advanced Subqueries	11
12	Views	12
13	Indexes	13
14	Transactions	14
15	Data Integrity and Constraints	15
16	Stored Procedures and Functions	16
17	Common Table Expressions (CTEs)	17
18	Window Functions	17
19	Prepared Statements	18
20	Performance Tuning and Query Optimization	19
21	PostgreSQL Expert Topics	20

1 Introduction to SQL

❖ Fun Fact

The `dvdrental` database is based on the Sakila sample database originally created by MySQL. It models a fictional DVD rental chain circa 2005–2006 with **600 films**, **599 customers**, and **16,044 rental transactions**. Think Blockbuster, but without the late-fee drama... or *with* it, depending on your SQL skills!

★ Concept: What is SQL? — The Big Picture

SQL (Structured Query Language) is the standard language for managing relational databases. Think of it as the *conversation* you have with PostgreSQL — you ask questions, it answers (sometimes slowly, but we'll fix that in Section 20).

In PostgreSQL, statements fall into five families:

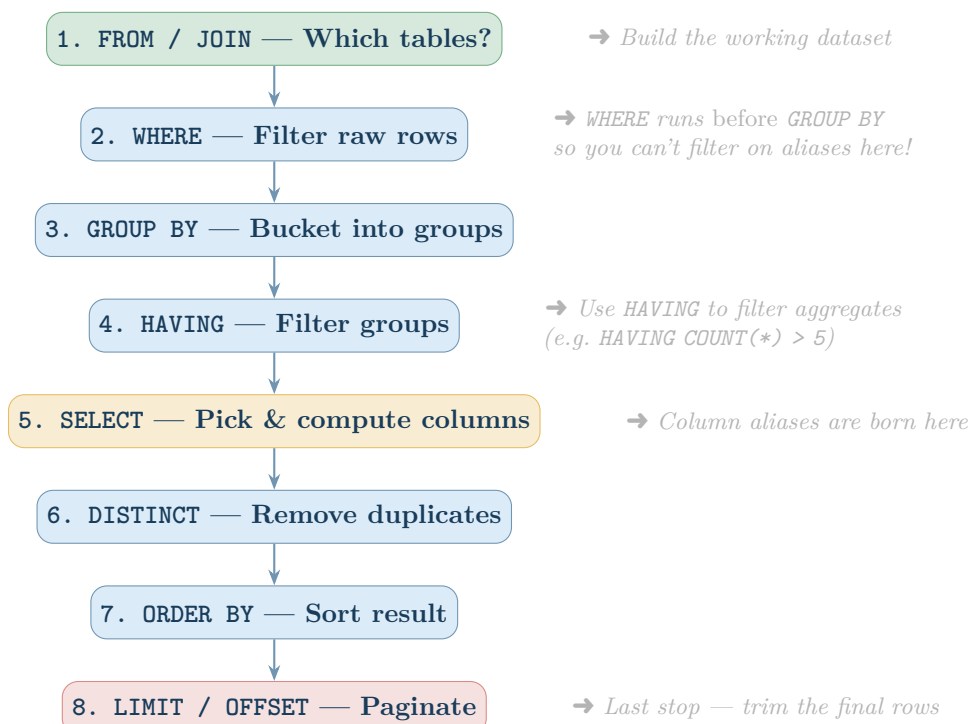
Family	Stands for	What it does
DDL	Data D efinition	Build / alter / drop tables & schemas
DML	Data M anipulation	INSERT, UPDATE, DELETE rows
DQL	Data Q uery	SELECT — the one you'll use 90% of the time
DCL	Data C ontrol	GRANT / REVOKE access
TCL	T ransaction Control	BEGIN, COMMIT, ROLLBACK

Key terms to know:

- **Primary Key (PK):** uniquely identifies each row (e.g., `film.film_id`)
- **Foreign Key (FK):** links one table to another (e.g., `rental.customer_id` → `customer`)
- **Schema:** a namespace grouping related objects inside a database
- **Relation:** the formal name for a table in relational theory

SQL Logical Query Execution Order

(This is the order PostgreSQL *evaluates* your query — not the order you *write* it!)



(!) Watch Out

Classic gotcha: You *cannot* reference a **SELECT** alias in **WHERE** or **HAVING** because those clauses execute *before* **SELECT**. Wrap the expression in a subquery or CTE instead.

Practice Questions

1. List all tables in `dvdrental` (`\dt` in `psql`). For each table, write one sentence describing its business role. EASY
2. What is the difference between `film` and `inventory`? Why does a single film have multiple `inventory` rows? EASY
3. Identify every primary key and foreign key in the `rental` table and draw the relationships on paper. EASY
4. Use `\d customer` in `psql` to inspect the `customer` table. Which columns could contain **NULL** and which cannot? EASY
5. Explain the purpose of the `film_actor` and `film_category` junction tables. What many-to-many relationship does each resolve? MEDIUM

★ Hint

Run `SELECT table_name FROM information_schema.tables WHERE table_schema = 'public';` to list all tables programmatically instead of using `\dt`.

❖ Fun Fact

The name **Sakila** (the origin of `dvdrental`) was chosen from a naming contest. It comes from a town in Tanzania suggested by a MySQL community member. So every time you `JOIN rental ON customer_id`, you're technically doing African geography trivia at the same time!

2 Data Definition Language (DDL)**★ Concept: DDL Commands**

DDL statements change the *structure* of the database, not its data.

- **CREATE TABLE** — defines a new table and its columns with data types and constraints
- **ALTER TABLE** — adds/removes/renames columns; adds constraints
- **DROP TABLE** — permanently removes the table and all its data
- **TRUNCATE TABLE** — removes all rows but keeps the table structure; faster than `DELETE`
- **SERIAL / BIGSERIAL** — shorthand for integer + auto-incrementing sequence

Practice Questions

1. Create a table `film_review` with: `review_id SERIAL PK, film_id INT FK → film, reviewer_name VARCHAR(100), stars INT, review_date DATE, comments TEXT`. EASY
2. Add a column `loyalty_points INT DEFAULT 0` to `customer`. Then rename it to `reward_points` and change its type to `NUMERIC(10,2)`. EASY

3. Create a table `film_promotion` with `SERIAL PK`, `film_id FK`, `discount_pct NUMERIC(5,2)`, and `valid_until DATE DEFAULT (CURRENT_DATE + 30)`. ● MEDIUM
4. Add a `NOT NULL` default value constraint to `film.rental_duration` ensuring no future row can have a `NULL` duration. ● MEDIUM
5. Explain the difference between `DROP TABLE film_review` and `TRUNCATE TABLE film_review`. When is each appropriate? ○ EASY

</> DDL Example — Create `film_review`

```
CREATE TABLE film_review (
  review_id    SERIAL PRIMARY KEY,
  film_id      INT NOT NULL
               REFERENCES film(film_id) ON DELETE CASCADE,
  reviewer_name VARCHAR(100) NOT NULL,
  stars        INT CHECK (stars BETWEEN 1 AND 5),
  review_date  DATE DEFAULT CURRENT_DATE,
  comments     TEXT
);
```

✓ Pro Tip

Always add `ON DELETE CASCADE` or `ON DELETE SET NULL` to foreign keys so PostgreSQL knows what to do when the parent row is removed.

3 Data Manipulation Language (DML)

★ Concept: DML Commands

DML statements change the *data* inside tables:

- **INSERT INTO ... VALUES / SELECT** — adds new rows
- **UPDATE ... SET ... WHERE** — modifies existing rows
- **DELETE FROM ... WHERE** — removes specific rows
- **RETURNING** — PostgreSQL extension: returns column values of the affected rows immediately after `INSERT / UPDATE / DELETE` without an extra `SELECT`

Practice Questions

1. Insert three new customer records (use `store_id=1`, `address_id=5`). Return the newly generated `customer_id` values using `RETURNING`. ○ EASY
2. A customer with `customer_id = 10` changed their email. Write the `UPDATE` and use `RETURNING email` to confirm the change. ○ EASY
3. All films with `rental_rate < 1.00` should receive a 20% price increase. Write a single `UPDATE` affecting multiple rows. ● MEDIUM
4. Delete all rentals where `return_date < NOW() - INTERVAL '5 years'` and capture the deleted `rental_id` and `customer_id` via `RETURNING`. ● MEDIUM
5. Insert a payment record for `rental_id = 100`, `customer_id = 10`, `staff_id = 1`, `amount = 4.99` and return the full inserted row. ○ EASY

</> DML Example — Bulk Update with RETURNING

```
-- Increase rental_rate for cheap films and see what changed
UPDATE film
SET   rental_rate = ROUND(rental_rate * 1.20, 2)
WHERE rental_rate < 1.00
RETURNING film_id, title, rental_rate AS new_rate;
```

(!) Watch Out

Always include a WHERE clause in UPDATE and DELETE or you will affect every row in the table. Test with SELECT first using the same WHERE condition.

4 Basic SQL Queries**★ Concept: Filtering and Selection**

The anatomy of a basic query:

```
SELECT columns FROM table WHERE condition ORDER BY col LIMIT n;
```

Key filtering tools:

- **LIKE / ILIKE** — pattern matching (% = any chars, _ = one char); ILIKE is case-insensitive
- **BETWEEN a AND b** — inclusive range (equivalent to >= a AND <= b)
- **IN (list)** — matches any value in the list
- **IS NULL / IS NOT NULL** — NULL cannot be tested with =
- **DISTINCT** — removes duplicate rows from the result set

Practice Questions

1. Retrieve the title, rating, and rental_rate of all films with rental_rate > 3.00. ○ EASY
2. Find all customers whose first_name starts with 'J' and whose active = true. ○ EASY
3. List every distinct rating value in the film table (G, PG, PG-13, R, NC-17). ○ EASY
4. Find films whose description contains the word 'Drama' (case-insensitive) and whose length is between 90 and 120 minutes. ● MEDIUM
5. Retrieve all rentals where return_date IS NULL (items still outstanding) together with the customer's email. ● MEDIUM

</> Example — ILIKE + BETWEEN + IS NULL

```
SELECT f.title, f.length, f.rating
FROM   film f
WHERE  f.description ILIKE '%drama%'
      AND f.length BETWEEN 90 AND 120;

-- Outstanding rentals
SELECT r.rental_id, c.email, r.rental_date
FROM   rental r
JOIN   customer c USING (customer_id)
WHERE  r.return_date IS NULL;
```

5 Sorting and Limiting Results

★ Concept: ORDER BY, LIMIT, OFFSET

- **ORDER BY col ASC|DESC** — sorts the result; multiple columns are comma-separated
- **LIMIT n** — returns at most n rows
- **OFFSET k** — skips the first k rows; used with LIMIT for pagination
- **Page formula:** page p , page size $s \Rightarrow \text{LIMIT } s \text{ OFFSET } (p - 1) \times s$
- **NULL ordering:** NULLs sort last in ASC by default; use NULLS FIRST or NULLS LAST to control

Practice Questions

1. Retrieve the top 10 most expensive films to rent, sorted by `rental_rate` descending. ○ EASY
2. List the 5 most recent rentals showing `rental_date`, `customer_id`, and `inventory_id`. ○ EASY
3. Implement page 3 of the film catalogue (page size = 10), sorted alphabetically by `title`.
● MEDIUM
4. Find the film with the second highest `replacement_cost` using ORDER BY + LIMIT + OFFSET.
● MEDIUM
5. Sort all customers by `last_name` ASC, then `first_name` ASC; return the first 15 rows. ○ EASY

★ Hint

For the **second highest** value, use ORDER BY `replacement_cost` DESC LIMIT 1 OFFSET 1. Be careful — if multiple films share the top value, OFFSET may skip the wrong row. Use a subquery or window function for a correct tie-safe result.

6 Functions and Expressions

★ Concept: Aggregate Functions

Aggregate functions collapse multiple rows into a single value:

Function	Purpose	DVD Rental Example
COUNT(*)	Total rows	Total number of rentals
COUNT(col)	Non-NULL count	Customers with an email
SUM(col)	Total	Total revenue
AVG(col)	Average	Avg rental rate per rating
MIN(col)	Minimum	Shortest film
MAX(col)	Maximum	Highest single payment

Practice Questions

1. Count the total number of films and the number of distinct `rating` values. ○ EASY
2. Calculate total revenue, average payment, and the single highest payment from the `payment` table.
○ EASY

3. Find the shortest and longest film by `length` (in minutes). EASY
4. Calculate the average `rental_rate` for each `rating` category (G, PG, PG-13, R, NC-17). MEDIUM
5. Count active vs inactive customers using `COUNT` with a `CASE` expression in a single query. MEDIUM

</> Example — CASE inside aggregate

```
SELECT
    COUNT(CASE WHEN active = true THEN 1 END) AS active_customers,
    COUNT(CASE WHEN active = false THEN 1 END) AS inactive_customers,
    COUNT(*) AS total_customers
FROM customer;
```

7 Grouping and Aggregating Data

★ Concept: GROUP BY and HAVING

- **GROUP BY** — collapses rows that share the same value(s) into one group; you can then apply aggregates per group
- **HAVING** — filters *after* grouping (like `WHERE` but for aggregated results)
- **Rule:** every column in `SELECT` must either appear in `GROUP BY` or be wrapped in an aggregate function
- **Execution order:** `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`

Practice Questions

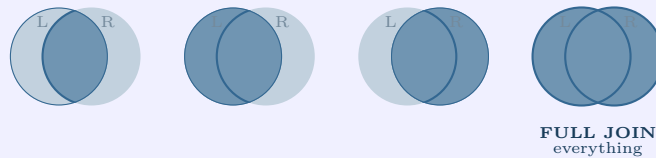
1. Count films per `rating` category; show only categories with more than 200 films. EASY
2. Find total revenue per `staff_id` and identify which staff member generated the most. MEDIUM
3. Detect duplicate customer email addresses using `GROUP BY email HAVING COUNT(*) > 1`. MEDIUM
4. Show the number of rentals processed each month (year + month from `rental_date`), sorted chronologically. MEDIUM
5. For each film `rating`, show the highest and lowest `rental_rate` side by side. EASY

</> Example — Monthly Rental Counts

```
SELECT
    DATE_TRUNC('month', rental_date) AS rental_month,
    COUNT(*) AS rental_count
FROM rental
GROUP BY rental_month
ORDER BY rental_month;
```

8 Joins

★ Concept: Join Types — Visualised



Join	Returns
INNER JOIN	Only rows with a match in both tables
LEFT JOIN	All rows from the left table; NULLs for non-matching right rows
RIGHT JOIN	All rows from the right table; NULLs for non-matching left rows
FULL JOIN	All rows from both tables; NULLs where no match exists
CROSS JOIN	Cartesian product — every left row paired with every right row

Tip: Use `USING(col)` when both tables share the same column name; use `ON t1.col = t2.col` otherwise.

Practice Questions

1. Join `rental` and `customer` to list each rental with the customer's full name and rental date.

○ EASY

2. Join `film`, `inventory`, and `rental` to show how many times each film has been rented.

● MEDIUM

3. Find all customers who have *never* made a rental using `LEFT JOIN` and `WHERE rental_id IS NULL`.

● MEDIUM

4. Produce a full film profile showing `title`, `category name`, and `language name` by joining `film`, `film_category`, `category`, and `language`.

● MEDIUM

5. Join `payment` and `rental` to show each payment with rental date and the number of days the item was kept (`return_date - rental_date`).

● MEDIUM

</> Example — Customers Who Never Rented

```
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
LEFT JOIN rental r USING (customer_id)
WHERE  r.rental_id IS NULL;
```

◆ Did You Know?

In the `dvdrental` database, **every** customer has at least one rental — there are no true non-renters hiding in the data. But in real-world scenarios, this “anti-join” pattern (`LEFT JOIN + WHERE right.pk IS NULL`) is one of the most useful tricks in SQL. It's faster than `NOT IN` and avoids the NULL-trap that `NOT IN` falls into when the subquery returns a NULL value!

9 Advanced Joins and Set Operations

★ Concept: SELF JOIN, CROSS JOIN, Set Operators

- **SELF JOIN** — joining a table to itself using two aliases; useful for hierarchies
- **UNION** — combines result sets and removes duplicates
- **UNION ALL** — combines result sets and *keeps* duplicates (faster)
- **INTERSECT** — returns only rows present in *both* result sets
- **EXCEPT** — returns rows in the first set that are *not* in the second
- All set operators require the same number of columns with compatible types

Practice Questions

1. Use a SELF JOIN on `staff` to show each staff member alongside their store manager (link via `store.manager_staff_id`). ▲ HARD
2. Use CROSS JOIN between film `rating` values and `store_id` to generate a pricing-matrix skeleton. ● MEDIUM
3. Use UNION to combine customer and staff emails into one deduplicated contact list with a `source` column. ● MEDIUM
4. Use INTERSECT to find `customer_ids` that appear in both `rental` and `payment`. ● MEDIUM
5. Use EXCEPT to find `customer_ids` in `rental` with no matching record in `payment` (rented but never paid). ● MEDIUM

</> Example — UNION with source label

```
SELECT email, 'customer' AS source FROM customer WHERE email IS NOT NULL
UNION
SELECT email, 'staff' AS source FROM staff WHERE email IS NOT NULL
ORDER BY email;
```

10 Subqueries and Nested Queries

★ Concept: Subquery Types

- **Scalar subquery** — returns exactly one value; can appear in SELECT or WHERE
 - **Row / Table subquery** — returns rows; used in FROM (derived table) or with IN
 - **EXISTS / NOT EXISTS** — returns TRUE if the subquery produces any row; short-circuits on first match (efficient)
 - **Correlated subquery** — references the outer query; re-evaluated for each outer row
- EXISTS vs IN:** EXISTS stops at the first match; IN materialises the full list first. For large result sets, EXISTS is typically faster.

Practice Questions

1. Find all films whose `rental_rate` exceeds the overall average `rental_rate`. ○ EASY

2. Use a subquery in the **FROM** clause to compute the average number of rentals per customer.

● MEDIUM

3. Use **EXISTS** to find customers who made at least one payment greater than \$7.00.

● MEDIUM

4. Use **NOT EXISTS** to find films that have never been added to inventory.

● MEDIUM

5. Use a scalar subquery in the **SELECT** clause to show each customer's name alongside their total rental count.

● MEDIUM

</> Example — EXISTS vs IN

```
-- EXISTS (stops at first match -- efficient)
SELECT c.customer_id, c.first_name
FROM   customer c
WHERE  EXISTS (
    SELECT 1 FROM payment p
    WHERE  p.customer_id = c.customer_id AND p.amount > 7.00
);

-- Equivalent with IN (materialises all matching customer_ids first)
SELECT customer_id, first_name
FROM   customer
WHERE  customer_id IN (
    SELECT customer_id FROM payment WHERE amount > 7.00
);
```

11 Advanced Subqueries

★ Concept: Correlated Subqueries

A **correlated subquery** references a column from the outer query. It runs once per outer row, making it potentially slow on large tables. Common uses:

- Row-by-row comparison against an aggregate (e.g., “above department average”)
- Finding the latest record per group
- Detecting duplicates

Optimisation: most correlated subqueries can be rewritten as a **JOIN + GROUP BY** or a window function, which PostgreSQL can execute more efficiently.

Practice Questions

1. Find customers whose total payments exceed the average total payment across all customers (correlated subquery).

▲ HARD

2. For each film category, find the film with the highest **rental_rate** using a correlated subquery.

▲ HARD

3. Find each customer's most recent rental date using a correlated subquery in the **WHERE** clause.

▲ HARD

4. Rewrite question 2 using a **JOIN** instead of a correlated subquery. Run **EXPLAIN ANALYZE** on both and compare.

▲ HARD

5. Use a correlated subquery to detect duplicate email addresses in `customer`, returning just the offending emails. ● MEDIUM

</> Example — Correlated vs JOIN rewrite

```
-- Correlated subquery (re-runs for each customer)
SELECT customer_id, SUM(amount) AS total
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > (SELECT AVG(total) FROM
                      (SELECT SUM(amount) AS total FROM payment
                        GROUP BY customer_id) sub);

-- Equivalent window-function approach (single pass)
SELECT DISTINCT customer_id, total
FROM (
  SELECT customer_id,
         SUM(amount) OVER (PARTITION BY customer_id) AS total,
         AVG(SUM(amount)) OVER () AS avg_total
  FROM payment
  GROUP BY customer_id
) t
WHERE total > avg_total;
```

12 Views

★ Concept: Views vs Materialized Views

	View	Materialized View
Storage	No (virtual)	Yes (physical table)
Freshness	Always current	Stale until refreshed
Speed	Runs query each time	Reads cached result
Refresh	Automatic	Manual (<code>REFRESH MATERIALIZED VIEW</code>)
Use case	Simplify complex joins	Heavy aggregation reports

Practice Questions

1. Create `v_customer_rentals`: each customer's full name, email, and total rental count. ○ EASY
2. Create `v_film_details` joining `film`, `language`, and `category` for a full film profile. ● MEDIUM
3. Create materialized view `mv_monthly_revenue` (total revenue per month). Refresh it, then query it. ● MEDIUM
4. Create `v_overdue_rentals`: all rentals where `return_date` IS NULL and rental is older than 7 days. ● MEDIUM
5. Why would you use a materialized view rather than a regular view for the monthly revenue report in a high-traffic rental system? What is the trade-off? ● MEDIUM

</> Example — Materialized View

```
CREATE MATERIALIZED VIEW mv_monthly_revenue AS
SELECT
    DATE_TRUNC('month', payment_date) AS revenue_month,
    SUM(amount)                      AS total_revenue,
    COUNT(*)                        AS payment_count
FROM payment
GROUP BY revenue_month
ORDER BY revenue_month;

-- Refresh (blocks reads; use CONCURRENTLY to avoid locking)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_monthly_revenue;
```

13 Indexes

★ Concept: Index Types in PostgreSQL

Type	Default?	Best for
B-Tree	Yes	Equality, range, ORDER BY on high-cardinality cols
Hash	No	Exact equality only
GIN	No	Full-text search, arrays, JSONB
GiST	No	Geometric/spatial data
BRIN	No	Very large tables with naturally ordered data (timestamps)
Partial	No	Index only a subset of rows (e.g., WHERE active = true)

Use EXPLAIN ANALYZE to verify that PostgreSQL actually uses an index — sometimes a sequential scan is cheaper for small tables.

Practice Questions

1. Create a B-Tree index on `customer.email`. Why is this column a good candidate? ○ EASY
2. Create a composite index on `rental(customer_id, rental_date)`. Describe which query patterns it will accelerate and which it will not. ● MEDIUM
3. Create a partial index on `rental(return_date)` WHERE `return_date IS NULL` to speed up overdue-rental lookups. ● MEDIUM
4. Run EXPLAIN ANALYZE on a film title search before and after adding an index on `film(title)`. What changes in the plan? ▲ HARD
5. Identify a low-cardinality column in the DVD rental schema that would *not* benefit from a B-Tree index and justify your answer. ● MEDIUM

</> Example — Partial Index for Overdue Rentals

```
-- Standard index would include ALL rentals; partial index is smaller
CREATE INDEX idx_rental_overdue
ON rental (rental_date)
WHERE return_date IS NULL;

-- This query now uses the partial index
EXPLAIN ANALYZE
```

```
SELECT rental_id, customer_id, rental_date
FROM rental
WHERE return_date IS NULL
AND rental_date < NOW() - INTERVAL '7 days';
```

◆ Did You Know?

A **partial index** can be *dramatically* smaller than a full-table index. In `dvdrental`, most rentals have a `return_date` set — the number of genuinely overdue (NULL) rentals at any moment is tiny. A partial index on just those rows fits in a fraction of the memory, making lookups lightning-fast. It's one of PostgreSQL's most underused superpowers!

14 Transactions

★ Concept: ACID Properties

Property	Meaning	DVD Rental Example
Atomicity	All or nothing	Insert rental & payment together; if payment fails, rental is rolled back
Consistency	Valid state before & after	FK constraints prevent an orphan rental
Isolation	Transactions don't interfere	Two clerks can rent different copies simultaneously
Durability	Committed data survives crash	WAL ensures a committed rental survives a power failure

Practice Questions

- Write a transaction that inserts a rental and a payment atomically. Roll back the whole unit if the payment insert fails. ● MEDIUM
- Simulate two concurrent sessions both trying to rent the last copy of a film. Explain what PostgreSQL does (row-level locking, MVCC). ▲ HARD
- Demonstrate all four ACID properties using the rental-and-payment scenario. ● MEDIUM
- Set the isolation level to `REPEATABLE READ` for a reporting session. When would this be important on the `payment` table? ▲ HARD
- Write a transaction that atomically transfers a staff member to a new store: update `staff.store_id` and `store.manager_staff_id`. ● MEDIUM

</> Example — Atomic Rental + Payment

```
BEGIN;

-- Step 1: create the rental
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 123, 10, 1)
RETURNING rental_id \gset -- psql variable assignment
```

```
-- Step 2: record payment (references the rental just created)
INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
VALUES (10, 1, :rental_id, 4.99, NOW());

COMMIT; -- both succeed, or ROLLBACK rolls back both
```

◆ Did You Know?

PostgreSQL uses **MVCC** (Multi-Version Concurrency Control) so readers never block writers and writers never block readers. When you run a **BEGIN** transaction, you get a consistent *snapshot* of the database — like freezing time. Other transactions can keep writing; you just can't see their changes until you start a new transaction. This is why **ROLLBACK** is so cheap: PostgreSQL simply discards your private snapshot rather than reversing physical writes!

15 Data Integrity and Constraints

★ Concept: Constraint Types

- **PRIMARY KEY** — unique + NOT NULL; one per table
- **FOREIGN KEY** — enforces referential integrity; can CASCADE, SET NULL, or RESTRICT on delete
- **UNIQUE** — no two rows may share the same value (NULLs are treated as distinct)
- **CHECK** — arbitrary boolean expression that every row must satisfy
- **NOT NULL** — column must always have a value
- **EXCLUSION** — advanced; ensures no two rows satisfy a given operator comparison

Practice Questions

1. Add a **CHECK** constraint to **film** ensuring **rental_duration** is between 1 and 30. ○ EASY
2. Add a **UNIQUE** constraint on **customer.email**. Show the error produced when you try to insert a duplicate. ○ EASY
3. Add **ON DELETE CASCADE** to the FK from **film_review.film_id** to **film.film_id**. Demonstrate it by deleting a film. ● MEDIUM
4. Add **NOT NULL** to **customer.email** on an existing table. What must you verify (and fix) before PostgreSQL will accept this change? ● MEDIUM
5. Design the full constraint set for a new **membership** table: PK, FK to **customer**, tier **CHECK IN ('bronze', 'silver', 'gold')**, **start_date NOT NULL**, **monthly_fee CHECK > 0**. ● MEDIUM

(!) Watch Out

Adding **NOT NULL** to an existing column fails if any current row contains **NULL** in that column. Always run **SELECT COUNT(*) FROM customer WHERE email IS NULL** first and clean up before altering the constraint.

16 Stored Procedures and Functions

★ Concept: Functions vs Procedures in PL/pgSQL

	FUNCTION	PROCEDURE
Returns value	Yes (RETURNS type)	No (use OUT params)
Called in SELECT	Yes	No (CALL only)
Transaction control	Cannot commit/rollback	Can commit/rollback
Use case	Computations, lookups	Multi-step workflows
PL/pgSQL blocks use DECLARE ...BEGIN ...EXCEPTION ...END.		

Practice Questions

1. Create function `get_customer_revenue(p_customer_id INT)` returning the total amount paid by that customer. ● MEDIUM
2. Create function `is_film_available(p_film_id INT, p_store_id INT)` returning `BOOLEAN` — true if any copy is not currently rented. ▲ HARD
3. Create procedure `process_return(p_rental_id INT)`: set `return_date = NOW()` and insert a late fee payment if overdue by more than 7 days. ▲ HARD
4. Add exception handling to `process_return` so it raises a descriptive error when the `rental_id` does not exist. ▲ HARD
5. Call both your function and procedure, verify the output, and explain when you would choose each. ● MEDIUM

</> Example — PL/pgSQL Function with Exception

```
CREATE OR REPLACE FUNCTION get_customer_revenue(p_customer_id INT)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COALESCE(SUM(amount), 0)
    INTO v_total
    FROM payment
    WHERE customer_id = p_customer_id;

    IF v_total = 0 THEN
        RAISE NOTICE 'No payments found for customer %', p_customer_id;
    END IF;

    RETURN v_total;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Error computing revenue: %', SQLERRM;
END;
$$ LANGUAGE plpgsql;

-- Call it
SELECT get_customer_revenue(10);
```

17 Common Table Expressions (CTEs)

★ Concept: CTEs — WITH Clause

A CTE defines a named, temporary result set usable within a single query.

- **Readability:** break complex queries into named logical steps
- **Reusability:** a CTE can be referenced multiple times in the same query
- **Recursive CTEs:** use `WITH RECURSIVE` for hierarchies, trees, and series generation
- **Performance note:** in PostgreSQL ≥ 12 a CTE is *not* automatically a fence — the planner can inline it unless you add `MATERIALIZED`

Practice Questions

1. Use a CTE to find the top 5 customers by rental count, then join back to show their full details. ● MEDIUM
2. Rewrite this as a CTE for readability: “Find all films rented more than 20 times.” ● MEDIUM
3. Use a recursive CTE to generate monthly date ranges from 2005-01-01 to 2006-12-01 and count rentals in each. ▲ HARD
4. Build a multi-CTE report: CTE 1 = revenue per customer, CTE 2 = overall average, final `SELECT` = customers above average. ▲ HARD
5. Compare CTE vs subquery performance for “top-rented film per category” using `EXPLAIN ANALYZE`. ★ EXPERT

</> Example — Recursive CTE: Monthly Series

```
WITH RECURSIVE months AS (
  SELECT DATE '2005-01-01' AS month_start
  UNION ALL
  SELECT month_start + INTERVAL '1 month'
  FROM months
  WHERE month_start < DATE '2006-12-01'
)
SELECT m.month_start,
       COUNT(r.rental_id) AS rentals
FROM   months m
LEFT JOIN rental r
      ON r.rental_date >= m.month_start
      AND r.rental_date < m.month_start + INTERVAL '1 month'
GROUP BY m.month_start
ORDER BY m.month_start;
```

18 Window Functions

★ Concept: Window Functions Anatomy

`function() OVER (PARTITION BY col ORDER BY col frame)`

- **ROW_NUMBER()** — unique sequential integer per partition; no ties
- **RANK()** — ties get the same rank; next rank jumps by the count of ties
- **DENSE_RANK()** — ties get the same rank; next rank is always +1

- **LAG(col, n) / LEAD(col, n)** — value *n* rows before/after current row
- **SUM() OVER (...ROWS BETWEEN ...)** — running/moving aggregates
- **FIRST_VALUE() / LAST_VALUE()** — first/last value in the window frame

Practice Questions

1. Use **ROW_NUMBER()** partitioned by **rating** to rank films by **rental_rate** within each rating group. ● MEDIUM
2. Use **RANK()** and **DENSE_RANK()** to rank customers by total payments. Show a scenario where they give different values. ● MEDIUM
3. Use **LAG(amount)** over **payment_date** to show each payment alongside the previous payment amount for the same customer. ▲ HARD
4. Calculate 3-month rolling revenue using **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW** over monthly totals. ▲ HARD
5. Use **FIRST_VALUE()** to show the most popular film (by rental count) in the same category next to every film. ★ EXPERT

</> Example — RANK vs DENSE_RANK

```
SELECT
  c.customer_id,
  c.first_name,
  SUM(p.amount)           AS total_paid,
  RANK()      OVER (ORDER BY SUM(p.amount) DESC) AS rank_with_gaps,
  DENSE_RANK() OVER (ORDER BY SUM(p.amount) DESC) AS rank_no_gaps
FROM   customer c
JOIN   payment p USING (customer_id)
GROUP BY c.customer_id, c.first_name
ORDER BY total_paid DESC
LIMIT 20;
```

The Leaderboard Challenge: Build a single query that produces a customer leaderboard showing: (1) their **RANK()** by total spend, (2) the **LAG()** of their previous month's revenue, (3) a **running_total** using **SUM() OVER (ORDER BY payment_date)**, and (4) a percentile via **PERCENT_RANK()**. No subqueries allowed — window functions only!

19 Prepared Statements

★ Concept: PREPARE / EXECUTE

Prepared statements compile a query template once; parameters are supplied at execution time.

- **SQL injection protection:** parameters are never interpreted as SQL — the structure is fixed
- **Plan caching:** after 5 executions PostgreSQL uses a generic plan (bypasses planning overhead)
- **Lifecycle:** **PREPARE** → **EXECUTE** (many times) → **DEALLOCATE**

- **Parameters:** referenced as \$1, \$2, ... in psql; as ? or named in application libraries

Practice Questions

1. Create prepared statement `get_films_by_rating` accepting a `rating` parameter; execute it for 'PG' and 'R'. ○ EASY
2. Create `customer_payment_history(customer_id INT)` and execute it for three different customers. ● MEDIUM
3. Demonstrate SQL injection protection: pass ' OR '1'='1 as the email parameter and show that a prepared statement neutralises it. ▲ HARD
4. Deallocate the prepared statements and explain when `DEALLOCATE ALL` is appropriate vs individual deallocation. ○ EASY
5. Explain PostgreSQL's "generic plan" threshold (5 executions). Why does this matter for connection-pool-heavy applications like a rental API? ▲ HARD

</> Example — Prepared Statement

```
PREPARE get_films_by_rating (TEXT) AS
  SELECT film_id, title, rental_rate
  FROM   film
  WHERE  rating = $1
  ORDER BY title;

EXECUTE get_films_by_rating('PG');
EXECUTE get_films_by_rating('R');

DEALLOCATE get_films_by_rating;
```

20 Performance Tuning and Query Optimization

★ Concept: The Optimization Toolkit

Tool	Purpose
EXPLAIN	Show the query plan without running the query
EXPLAIN ANALYZE	Execute the query and show actual vs estimated costs
VACUUM	Reclaim storage from dead tuples; update visibility map
ANALYZE	Update table statistics used by the query planner
VACUUM ANALYZE	Both in one command
pg_stat_statements	Extension that records cumulative query statistics
Key plan nodes to watch: Seq Scan (no index), Index Scan , Index Only Scan (fastest), Hash Join , Nested Loop , Merge Join .	

Practice Questions

1. Run `EXPLAIN ANALYZE` on a join of `rental`, `customer`, and `payment`. Identify Seq Scans and propose fixes. ● MEDIUM

2. Profile the monthly revenue report. Identify the bottleneck and propose whether an index, materialized view, or query rewrite would help most. ▲ HARD
3. Run `VACUUM ANALYZE` on the `rental` table. Why is this critical after a large batch delete? ● MEDIUM
4. Enable `pg_stat_statements` and query it to find the top 5 slowest queries by `mean_exec_time`. ▲ HARD
5. Rewrite a correlated subquery that finds the latest rental per customer as a `LATERAL` join. Compare execution plans. ★ EXPERT

</> Example — pg_stat_statements Top 5 Slow Queries

```
-- Enable once (superuser required)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Find the 5 slowest queries by mean execution time
SELECT LEFT(query, 80) AS query_snippet,
       calls,
       ROUND(mean_exec_time::NUMERIC, 2) AS avg_ms,
       ROUND(total_exec_time::NUMERIC, 2) AS total_ms
FROM   pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 5;
```

21 PostgreSQL Expert Topics

★ Concept: Expert Concepts at a Glance

- **MVCC** (Multi-Version Concurrency Control) — each transaction sees a snapshot; writers never block readers
- **WAL** (Write-Ahead Log) — durability mechanism; changes are logged before they are applied
- **Partitioning** — `RANGE`, `LIST`, or `HASH` split a large table into child tables for pruning and parallelism
- **JSONB** — binary JSON stored in PostgreSQL; supports GIN indexing for fast key/value search
- **GIN index** — generalised inverted index for full-text search (`tsvector`) and JSONB
- **BRIN index** — block range index; tiny footprint for append-only timestamp columns in huge tables
- **Autovacuum** — background daemon that runs `VACUUM`/`ANALYZE` automatically; tune aggressiveness with `autovacuum_vacuum_scale_factor`

Practice Questions

1. Explain how MVCC lets two clerks query `rental` simultaneously without blocking each other. What are “dead tuples” and why do they accumulate? ▲ HARD
2. Partition the `payment` table by `payment_date` (`RANGE`, one partition per year: 2005, 2006, default). Demonstrate partition pruning with `EXPLAIN`. ★ EXPERT
3. The `film` table has a `fulltext tsvector` column. Create a GIN index on it and write a full-text search for films containing ‘astronaut’. ▲ HARD

4. Design an audit log table for every rental and payment event. Use a BRIN index on the timestamp column. Justify why BRIN is better than B-Tree here. ★ EXPERT
5. A query on a billion-row `rental_archive` table takes 8 minutes. Walk through your complete optimisation strategy: partitioning, indexes, materialized views, and query rewriting. ★ EXPERT

</> Example — Full-Text Search with GIN

```
-- GIN index on existing tsvector column
CREATE INDEX idx_film_fulltext ON film USING GIN (fulltext);

-- Full-text search query
SELECT film_id, title
FROM film
WHERE fulltext @@ PLAINTO_TSQUERY('english', 'astronaut')
ORDER BY ts_rank(fulltext, PLAINTO_TSQUERY('english', 'astronaut')) DESC;

-- Range-partitioned payment table
CREATE TABLE payment_partitioned (
    LIKE payment INCLUDING ALL
) PARTITION BY RANGE (payment_date);

CREATE TABLE payment_2005
    PARTITION OF payment_partitioned
    FOR VALUES FROM ('2005-01-01') TO ('2006-01-01');

CREATE TABLE payment_2006
    PARTITION OF payment_partitioned
    FOR VALUES FROM ('2006-01-01') TO ('2007-01-01');

CREATE TABLE payment_default
    PARTITION OF payment_partitioned DEFAULT;
```

✓ Pro Tip

Use `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` and paste the output into explain.dalibo.com for a beautiful visual execution plan — invaluable for diagnosing slow queries in production.

The Billion-Row Gauntlet: Imagine the `rental` table has **1 billion rows** spanning 10 years. Design a complete strategy covering: range partitioning by year, BRIN index on `rental_date`, a GIN index on a JSONB metadata column, a nightly-refreshed materialized view for reporting, and connection-pooling via PgBouncer. Write the DDL. Then explain to a non-DBA colleague why this is better than just “adding more RAM.”

Quick Reference Card — Your SQL Cheat Sheet

Stuck? Glance here. Unstuck? Write a CTE.

Topic	Key Command / Function
Inspect table	<code>\d tablename</code>
Execution plan	<code>EXPLAIN ANALYZE SELECT ...</code>
Overdue rentals	<code>WHERE return_date IS NULL</code>
Running total	<code>SUM(amt) OVER (ORDER BY date)</code>
Latest per group	<code>ROW_NUMBER() OVER (PARTITION BY...)</code>
Full-text search	<code>fulltext @@ PLAINTO_TSQUERY('word')</code>
Refresh MV	<code>REFRESH MATERIALIZED VIEW CONCURRENTLY mv</code>
Slow queries	<code>SELECT * FROM pg_stat_statements ORDER BY mean_exec_time DESC</code>
Cleanup	<code>VACUUM ANALYZE tablename</code>
Partition prune	<code>EXPLAIN</code> on range-partitioned table

21 Topics • 105 Questions • SQL Hints & Answer Guides
Muhammad Usman • Data Engineering & Cloud Architecture

“The best time to add an index was yesterday. The second best time is after reading Section 10.”